

```
1 pid = Fork();
```

我们将在本书剩余的部分中都使用错误处理包装函数。它们能够保持代码示例简洁，而又不会给你错误的假象，认为允许忽略错误检查。注意，当在本书中谈到系统级函数时，我们总是用它们的小写字母的基本名字来引用它们，而不是用它们大写的包装函数名来引用。

关于 Unix 错误处理以及本书中使用的错误处理包装函数的讨论，请参见附录 A。包装函数定义在一个叫做 csapp.c 的文件中，它们的原型定义在一个叫做 csapp.h 的头文件中；可以从 CS:APP 网站上在线地得到这些代码。

8.4 进程控制

Unix 提供了大量从 C 程序中操作进程的系统调用。这一节将描述这些重要的函数，并举例说明如何使用它们。

8.4.1 获取进程 ID

每个进程都有一个唯一的正数(非零)进程 ID(PID)。getpid 函数返回调用进程的 PID。getppid 函数返回它的父进程的 PID(创建调用进程的进程)。

```
#include <sys/types.h>
#include <unistd.h>

pid_t getpid(void);
pid_t getppid(void);
```

返回：调用者或其父进程的 PID。

getpid 和 getppid 函数返回一个类型为 pid_t 的整数值，在 Linux 系统上它在 types.h 中被定义为 int。

8.4.2 创建和终止进程

从程序员的角度，我们可以认为进程总是处于下面三种状态之一：

- 运行。进程要么在 CPU 上执行，要么在等待被执行且最终会被内核调度。
- 停止。进程的执行被挂起(suspended)，且不会被调度。当收到 SIGSTOP、SIGTSTP、SIGTTIN 或者 SIGTTOU 信号时，进程就停止，并且保持停止直到它收到一个 SIGCONT 信号，在这个时刻，进程再次开始运行。(信号是一种软件中断的形式，将在 8.5 节中详细描述。)
- 终止。进程永远地停止了。进程会因为三种原因终止：1) 收到一个信号，该信号的默认行为是终止进程，2) 从主程序返回，3) 调用 exit 函数。

```
#include <stdlib.h>

void exit(int status);
```

该函数不返回。

exit 函数以 status 退出状态来终止进程(另一种设置退出状态的方法是从主程序中返回一个整数值)。

父进程通过调用 fork 函数创建一个新的运行的子进程。

```
#include <sys/types.h>
#include <unistd.h>

pid_t fork(void);
```

返回：子进程返回 0，父进程返回子进程的 PID，如果出错，则为 -1。

新创建的子进程几乎但不完全与父进程相同。子进程得到与父进程用户级虚拟地址空间相同的(但是独立的)一份副本，包括代码和数据段、堆、共享库以及用户栈。子进程还获得与父进程任何打开文件描述符相同的副本，这就意味着当父进程调用 `fork` 时，子进程可以读写父进程中打开的任何文件。父进程和新创建的子进程之间最大的区别在于它们有不同的 PID。

`fork` 函数是有趣的(也常常令人迷惑)，因为它只被调用一次，却会返回两次：一次是在调用进程(父进程)中，一次是在新创建的子进程中。在父进程中，`fork` 返回子进程的 PID。在子进程中，`fork` 返回 0。因为子进程的 PID 总是为非零，返回值就提供一个明确的方法来分辨程序是在父进程还是在子进程中执行。

图 8-15 展示了一个使用 `fork` 创建子进程的父进程的示例。当 `fork` 调用在第 6 行返回时，在父进程和子进程中 `x` 的值都为 1。子进程在第 8 行加一并输出它的 `x` 的副本。相似地，父进程在第 13 行减一并输出它的 `x` 的副本。

```

1  int main()
2  {
3      pid_t pid;
4      int x = 1;
5
6      pid = Fork();
7      if (pid == 0) { /* Child */
8          printf("child : x=%d\n", ++x);
9          exit(0);
10     }
11
12     /* Parent */
13     printf("parent: x=%d\n", --x);
14     exit(0);
15 }
```

code/ecf/fork.c

code/ecf/fork.c

图 8-15 使用 `fork` 创建一个新进程

当在 Unix 系统上运行这个程序时，我们得到下面的结果：

```
linux> ./fork
parent: x=0
child : x=2
```

这个简单的例子有一些微妙的方面。

- 调用一次，返回两次。`fork` 函数被父进程调用一次，但是却返回两次——一次是返回到父进程，一次是返回到新创建的子进程。对于只创建一个子进程的程序来说，这还是相当简单直接的。但是具有多个 `fork` 实例的程序可能就会令人迷惑，需要仔细地推敲了。
- 并发执行。父进程和子进程是并发运行的独立进程。内核能够以任意方式交替执行它们的逻辑控制流中的指令。在我们的系统上运行这个程序时，父进程先完成它的 `printf` 语句，然后是子进程。然而，在另一个系统上可能正好相反。一般而言，

作为程序员，我们决不能对不同进程中指令的交替执行做任何假设。

- 相同但是独立的地址空间。如果能够在 `fork` 函数在父进程和子进程中返回后立即暂停这两个进程，我们会看到两个进程的地址空间都是相同的。每个进程有相同的用户栈、相同的本地变量值、相同的堆、相同的全局变量值，以及相同的代码。因此，在我们的示例程序中，当 `fork` 函数在第 6 行返回时，本地变量 `x` 在父进程和子进程中都为 1。然而，因为父进程和子进程是独立的进程，它们都有自己的私有地址空间。后面，父进程和子进程对 `x` 所做的任何改变都是独立的，不会反映在另一个进程的内存中。这就是为什么当父进程和子进程调用它们各自的 `printf` 语句时，它们中的变量 `x` 会有不同的值。
- 共享文件。当运行这个示例程序时，我们注意到父进程和子进程都把它们输出显示在屏幕上。原因是子进程继承了父进程所有的打开文件。当父进程调用 `fork` 时，`stdout` 文件是打开的，并指向屏幕。子进程继承了这个文件，因此它的输出也是指向屏幕的。

如果你是第一次学习 `fork` 函数，画进程图通常会有所帮助，进程图是刻画程序语句的偏序的一种简单的前趋图。每个顶点 a 对应于一条程序语句的执行。有向边 $a \rightarrow b$ 表示语句 a 发生在语句 b 之前。边上可以标记出一些信息，例如一个变量的当前值。对应于 `printf` 语句的顶点可以标记上 `printf` 的输出。每张图从一个顶点开始，对应于调用 `main` 的父进程。这个顶点没有入边，并且只有一个出边。每个进程的顶点序列结束于一个对应于 `exit` 调用的顶点。这个顶点只有一条入边，没有出边。

例如，图 8-16 展示了图 8-15 中示例程序的进程图。初始时，父进程将变量 `x` 设置为 1。父进程调用 `fork`，创建一个子进程，它在自己的私有地址空间中与父进程并发执行。

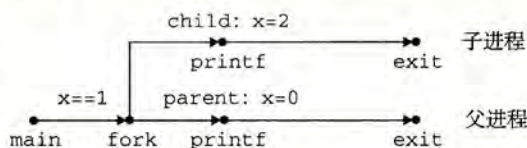


图 8-16 图 8-15 中示例程序的进程图

对于运行在单处理器上的程序，对应进程图中所有顶点的拓扑排序(topological sort)表示程序中语句的一个可行的全序排列。下面是一个理解拓扑排序概念的简单方法：给定进程图中顶点的一个排列，把顶点序列从左到右写成一行，然后画出每条有向边。排列是一个拓扑排序，当且仅当画出的每条边的方向都是从左往右的。因此，在图 8-15 的示例程序中，父进程和子进程的 `printf` 语句可以以任意先后顺序执行，因为每种顺序都对应于图顶点的某种拓扑排序。

进程图特别有助于理解带有嵌套 `fork` 调用的程序。例如，图 8-17 中的程序源码中两次调用了 `fork`。对应的进程图可帮助我们看清这个程序运行了四个进程，每个都调用了一次 `printf`，这些 `printf` 可以以任意顺序执行。

```

1  int main()
2  {
3      Fork();
4      Fork();
5      printf("hello\n");
6      exit(0);
7  }
```

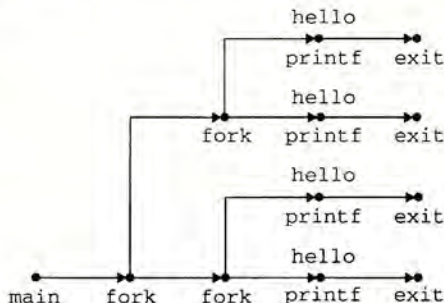


图 8-17 嵌套 `fork` 的进程图

练习题 8.2 考虑下面的程序：

```

1  int main()
2  {
3      int x = 1;
4
5      if (Fork() == 0)
6          printf("p1: x=%d\n", ++x);
7      printf("p2: x=%d\n", --x);
8      exit(0);
9  }
```

code/ecf/forkprob0.c

code/ecf/forkprob0.c

- A. 子进程的输出是什么？
- B. 父进程的输出是什么？

8.4.3 回收子进程

当一个进程由于某种原因终止时，内核并不是立即把它从系统中清除。相反，进程被保持在一种已终止的状态中，直到被它的父进程回收(reaped)。当父进程回收已终止的子进程时，内核将子进程的退出状态传递给父进程，然后抛弃已终止的进程，从此时开始，该进程就不存在了。一个终止了但还未被回收的进程称为僵死进程(zombie)。

旁注 为什么已终止的子进程被称为僵死进程？

在民间传说中，僵尸是活着的尸体，一种半生半死的实体。僵死进程已经终止了，而内核仍保留着它的某些状态直到父进程回收它为止，从这个意义上说它们是类似的。

如果一个父进程终止了，内核会安排 init 进程成为它的孤儿进程的养父。init 进程的 PID 为 1，是在系统启动时由内核创建的，它不会终止，是所有进程的祖先。如果父进程没有回收它的僵死子进程就终止了，那么内核会安排 init 进程去回收它们。不过，长时间运行的程序，比如 shell 或者服务器，总是应该回收它们的僵死子进程。即使僵死子进程没有运行，它们仍然消耗系统的内存资源。

一个进程可以通过调用 waitpid 函数来等待它的子进程终止或者停止。

```

#include <sys/types.h>
#include <sys/wait.h>

pid_t waitpid(pid_t pid, int *statusp, int options);
```

返回：如果成功，则为子进程的 PID，如果 WNOHANG，则为 0，如果其他错误，则为 -1。

waitpid 函数有点复杂。默认情况下(当 options=0 时)，waitpid 挂起调用进程的执行，直到它的等待集合(wait set)中的一个子进程终止。如果等待集合中的一个进程在刚调用的时刻就已经终止了，那么 waitpid 就立即返回。在这两种情况中，waitpid 返回导致 waitpid 返回的已终止子进程的 PID。此时，已终止的子进程已经被回收，内核会从系统中删除掉它的所有痕迹。

1. 判定等待集合的成员

等待集合的成员是由参数 pid 来确定的：

- 如果 `pid>0`，那么等待集合就是一个单独的子进程，它的进程 ID 等于 `pid`。
 - 如果 `pid=-1`，那么等待集合就是由父进程所有的子进程组成的。
- `waitpid` 函数还支持其他类型的等待集合，包括 Unix 进程组，对此我们将不做讨论。

2. 修改默认行为

可以通过将 `options` 设置为常量 `WNOHANG`、`WUNTRACED` 和 `WCONTINUED` 的各种组合来修改默认行为：

- **WNOHANG**：如果等待集合中的任何子进程都还没有终止，那么就立即返回（返回值为 0）。默认的行为是挂起调用进程，直到有子进程终止。在等待子进程终止的同时，如果还想做些有用的工作，这个选项会有用。
- **WUNTRACED**：挂起调用进程的执行，直到等待集合中的一个进程变成已终止或者被停止。返回的 PID 为导致返回的已终止或被停止子进程的 PID。默认的行为是只返回已终止的子进程。当你想要检查已终止和被停止的子进程时，这个选项会有用。
- **WCONTINUED**：挂起调用进程的执行，直到等待集合中一个正在运行的进程终止或等待集合中一个被停止的进程收到 `SIGCONT` 信号重新开始执行。（8.5 节会解释这些信号。）

可以用或运算把这些选项组合起来。例如：

- **WNOHANG | WUNTRACED**：立即返回，如果等待集合中的子进程都没有被停止或终止，则返回值为 0；如果有一个停止或终止，则返回值为该子进程的 PID。

3. 检查已回收子进程的退出状态

如果 `statusp` 参数是非空的，那么 `waitpid` 就会在 `status` 中放上关于导致返回的子进程的状态信息，`status` 是 `statusp` 指向的值。`wait.h` 头文件定义了解释 `status` 参数的几个宏：

- **WIFEXITED(status)**：如果子进程通过调用 `exit` 或者一个返回(`return`)正常终止，就返回真。
- **WEXITSTATUS(status)**：返回一个正常终止的子进程的退出状态。只有在 `WIFEXITED()` 返回为真时，才会定义这个状态。
- **WIFSIGNALED(status)**：如果子进程是因为一个未被捕获的信号终止的，那么就返回真。
- **WTERMSIG(status)**：返回导致子进程终止的信号的编号。只有在 `WIFSIGNALED()` 返回为真时，才定义这个状态。
- **WIFSTOPPED(status)**：如果引起返回的子进程当前是停止的，那么就返回真。
- **WSTOPSIG(status)**：返回引起子进程停止的信号的编号。只有在 `WIFSTOPPED()` 返回为真时，才定义这个状态。
- **WIFCONTINUED(status)**：如果子进程收到 `SIGCONT` 信号重新启动，则返回真。

4. 错误条件

如果调用进程没有子进程，那么 `waitpid` 返回 `-1`，并且设置 `errno` 为 `ECHILD`。如果 `waitpid` 函数被一个信号中断，那么它返回 `-1`，并设置 `errno` 为 `EINTR`。

旁注 和 Unix 函数相关的常量

像 `WNOHANG` 和 `WUNTRACED` 这样的常量是由系统头文件定义的。例如，`WNOHANG` 和 `WUNTRACED` 是由 `wait.h` 头文件(间接)定义的：


```
/* Bits in the third argument to 'waitpid'. */
#define WNOHANG 1 /* Don't block waiting. */
#define WUNTRACED 2 /* Report status of stopped children. */
```

为了使用这些常量，必须在代码中包含 `wait.h` 头文件：

```
#include <sys/wait.h>
```

每个 Unix 函数的 man 页列出了无论何时你在代码中使用那个函数都要包含的头文件。同时，为了检查诸如 `ECHILD` 和 `EINTR` 之类的返回代码，你必须包含 `errno.h`。为了简化代码示例，我们包含了一个称为 `csapp.h` 的头文件，它包括了本书中使用的所有函数的头文件。`csapp.h` 头文件可以从 CS: APP 网站在线获得。

 **练习题 8.3** 列出下面程序所有可能的输出序列：

```
1  int main()
2  {
3      if (Fork() == 0) {
4          printf("a"); fflush(stdout);
5      }
6      else {
7          printf("b"); fflush(stdout);
8          waitpid(-1, NULL, 0);
9      }
10     printf("c"); fflush(stdout);
11     exit(0);
12 }
```

code/ecf/waitprob0.c

code/ecf/waitprob0.c

5. wait 函数

`wait` 函数是 `waitpid` 函数的简单版本：

```
#include <sys/types.h>
#include <sys/wait.h>

pid_t wait(int *statusp);
```

返回：如果成功，则为子进程的 PID，如果出错，则为 -1。

调用 `wait(&status)` 等价于调用 `waitpid(-1, &status, 0)`。

6. 使用 `waitpid` 的示例

因为 `waitpid` 函数有些复杂，看几个例子会有所帮助。图 8-18 展示了一个程序，它使用 `waitpid`，不按照特定的顺序等待它的所有 N 个子进程终止。在第 11 行，父进程创建 N 个子进程，在第 12 行，每个子进程以一个唯一的退出状态退出。在我们继续讲解之前，请确认你已经理解为什么每个子进程会执行第 12 行，而父进程不会。

在第 15 行，父进程用 `waitpid` 作为 `while` 循环的测试条件，等待它所有的子进程终止。因为第一个参数是 -1，所以对 `waitpid` 的调用会阻塞，直到任意一个子进程终止。在每个子进程终止时，对 `waitpid` 的调用会返回，返回值为该子进程的非零的 PID。第 16 行检查子进程的退出状态。如果子进程是正常终止的——在此是以调用 `exit` 函数终止的——那么父进程就提取出退出状态，把它输出到 `stdout` 上。

```

1  #include "csapp.h"
2  #define N 2
3
4  int main()
5  {
6      int status, i;
7      pid_t pid;
8
9      /* Parent creates N children */
10     for (i = 0; i < N; i++)
11         if ((pid = Fork()) == 0) /* Child */
12             exit(100+i);
13
14     /* Parent reaps N children in no particular order */
15     while ((pid = waitpid(-1, &status, 0)) > 0) {
16         if (WIFEXITED(status))
17             printf("child %d terminated normally with exit status=%d\n",
18                   pid, WEXITSTATUS(status));
19         else
20             printf("child %d terminated abnormally\n", pid);
21     }
22
23     /* The only normal termination is if there are no more children */
24     if (errno != ECHILD)
25         unix_error("waitpid error");
26
27     exit(0);
28 }

```

code/ecf/waitpid1.c

图 8-18 使用 waitpid 函数不按照特定的顺序回收僵死子进程

当回收了所有的子进程之后，再调用 waitpid 就返回 -1，并且设置 errno 为 ECHILD。第 24 行检查 waitpid 函数是正常终止的，否则就输出一个错误消息。在我们的 Linux 系统上运行这个程序时，它产生如下输出：

```

linux> ./waitpid1
child 22966 terminated normally with exit status=100
child 22967 terminated normally with exit status=101

```

注意，程序不会按照特定的顺序回收子进程。子进程回收的顺序是这台特定的计算机系统的属性。在另一个系统上，甚至在同一个系统上再执行一次，两个子进程都可能以相反的顺序被回收。这是非确定性行为的一个示例，这种非确定性行为使得对并发进行推理非常困难。两种可能的结果都同样是正确的，作为一个程序员，你绝不可以假设总是会出现某一个结果，无论多么不可能出现另一个结果。唯一正确的假设是每一个可能的结果都同样可能出现。

图 8-19 展示了一个简单的改变，它消除了这种不确定性，按照父进程创建子进程的相同顺序来回收这些子进程。在第 11 行中，父进程按照顺序存储了它的子进程的 PID，然后通过用适当的 PID 作为第一个参数来调用 waitpid，按照同样的顺序来等待每个子进程。

code/ecf/waitpid2.c

```

1  #include "csapp.h"
2  #define N 2
3
4  int main()
5  {
6      int status, i;
7      pid_t pid[N], retpid;
8
9      /* Parent creates N children */
10     for (i = 0; i < N; i++)
11         if ((pid[i] = Fork()) == 0) /* Child */
12             exit(100+i);
13
14     /* Parent reaps N children in order */
15     i = 0;
16     while ((retpid = waitpid(pid[i++], &status, 0)) > 0) {
17         if (WIFEXITED(status))
18             printf("child %d terminated normally with exit status=%d\n",
19                   retpid, WEXITSTATUS(status));
20         else
21             printf("child %d terminated abnormally\n", retpid);
22     }
23
24     /* The only normal termination is if there are no more children */
25     if (errno != ECHILD)
26         unix_error("waitpid error");
27
28     exit(0);
29 }

```

code/ecf/waitpid2.c

图 8-19 使用 waitpid 按照创建子进程的顺序来回收这些僵死子进程



练习题 8.4 考虑下面的程序：

code/ecf/waitprobl.c

```

1  int main()
2  {
3      int status;
4      pid_t pid;
5
6      printf("Hello\n");
7      pid = Fork();
8      printf("%d\n", !pid);
9      if (pid != 0) {
10         if (waitpid(-1, &status, 0) > 0) {
11             if (WIFEXITED(status) != 0)
12                 printf("%d\n", WEXITSTATUS(status));
13         }
14     }
15     printf("Bye\n");
16     exit(2);
17 }

```

code/ecf/waitprobl.c

- A. 这个程序会产生多少输出行?
B. 这些输出行的一种可能的顺序是什么?

8.4.4 让进程休眠

sleep 函数将一个进程挂起一段指定的时间。

```
#include <unistd.h>

unsigned int sleep(unsigned int secs);
```

返回：还要休眠的秒数。

如果请求的时间量已经到了，sleep 返回 0，否则返回还剩下的要休眠的秒数。后一种情况是可能的，如果因为 sleep 函数被一个信号中断而过早地返回。我们将在 8.5 节中详细讨论信号。

我们会发现另一个很有用的函数是 pause 函数，该函数让调用函数休眠，直到该进程收到一个信号。

```
#include <unistd.h>

int pause(void);
```

总是返回 -1。

 **练习题 8.5** 编写一个 sleep 的包装函数，叫做 snooze，带有下面的接口：

```
unsigned int snooze(unsigned int secs);
```

snooze 函数和 sleep 函数的行为完全一样，除了它会打印出一条消息来描述进程实际休眠了多长时间：

```
Slept for 4 of 5 secs.
```

8.4.5 加载并运行程序

execve 函数在当前进程的上下文中加载并运行一个新程序。

```
#include <unistd.h>

int execve(const char *filename, const char *argv[],
           const char *envp[]);
```

如果成功，则不返回，如果错误，则返回 -1。

execve 函数加载并运行可执行目标文件 filename，且带参数列表 argv 和环境变量列表 envp。只有当出现错误时，例如找不到 filename，execve 才会返回到调用程序。所以，与 fork 一次调用返回两次不同，execve 调用一次并从不返回。

参数列表是用图 8-20 中的数据结构表示的。argv 变量指向一个以 null 结尾的指针数组，其中每个指针都指向一个参数字符串。按照惯例，argv[0] 是可执行目标文件的名字。环境变量的列表是由一个类似的数据结构表示的，如图 8-21 所示。envp 变量指向一个以 null 结尾的指针数组，其中每个指针指向一个环境变量字符串，每个串都是形如 “name=value” 的名字-值对。

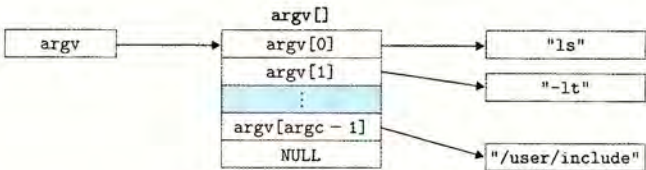


图 8-20 参数列表的组织结构

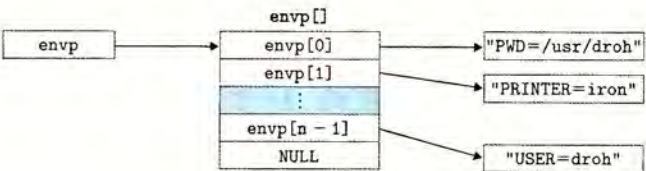


图 8-21 环境变量列表的组织结构

在 `execve` 加载了 `filename` 之后，它调用 7.9 节中描述的启动代码。启动代码设置栈，并将控制传递给新程序的主函数，该主函数有如下形式的原型

```
int main(int argc, char **argv, char **envp);
```

或者等价的

```
int main(int argc, char *argv[], char *envp[]);
```

当 `main` 开始执行时，用户栈的组织结构如图 8-22 所示。让我们从栈底（高地址）往栈顶（低地址）依次看一看。首先是参数和环境字符串。栈往上紧随其后的是以 `null` 结尾的指针数组，其中每个指针都指向栈中的一个环境变量字符串。全局变量 `environ` 指向这些指针中的第一个 `envp[0]`。紧随环境变量数组之后的是以 `null` 结尾的 `argv[]` 数组，其中每个元素都指向栈中的一个参数字符串。在栈的顶部是系统启动函数 `libc_start_main`（见 7.9 节）的栈帧。

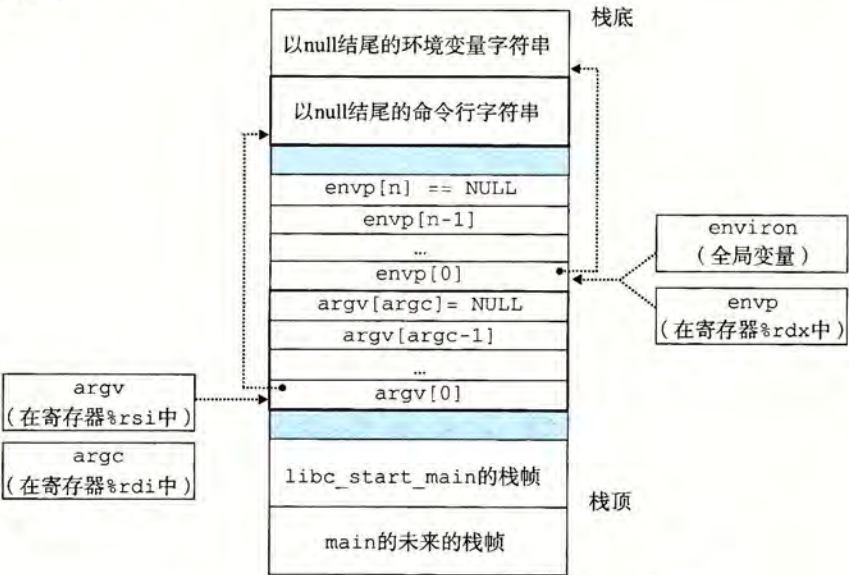


图 8-22 一个新程序开始时，用户栈的典型组织结构

main 函数有 3 个参数：1) argc，它给出 argv[] 数组中非空指针的数量，2) argv，指向 argv[] 数组中的第一个条目，3) envp，指向 envp[] 数组中的第一个条目。

Linux 提供了几个函数来操作环境数组：

```
#include <stdlib.h>
```

```
char *getenv(const char *name);
```

返回：若存在则为指向 name 的指针，若无匹配的，则为 NULL。

getenv 函数在环境数组中搜索字符串 “name=value”。如果找到了，它就返回一个指向 value 的指针，否则它就返回 NULL。

```
#include <stdlib.h>
```

```
int setenv(const char *name, const char *newvalue, int overwrite);
```

返回：若成功则为 0，若错误则为 -1。

```
void unsetenv(const char *name);
```

返回：无。

如果环境数组包含一个形如 “name=oldvalue” 的字符串，那么 unsetenv 会删除它，而 setenv 会用 newvalue 代替 oldvalue，但是只有在 overwrite 非零时才会这样。如果 name 不存在，那么 setenv 就把 “name=newvalue” 添加到数组中。

旁注 程序与进程

这是一个适当的地方，停下来，确认一下你理解了程序和进程之间的区别。程序是一堆代码和数据；程序可以作为目标文件存在于磁盘上，或者作为段存在于地址空间中。进程是执行中程序的一个具体的实例；程序总是运行在某个进程的上下文中。如果你想要理解 fork 和 execve 函数，理解这个差异是很重要的。fork 函数在新的子进程中运行相同的程序，新的子进程是父进程的一个复制品。execve 函数在当前进程的上下文中加载并运行一个新的程序。它会覆盖当前进程的地址空间，但并没有创建一个新进程。新的程序仍然有相同的 PID，并且继承了调用 execve 函数时已打开的所有文件描述符。

 **练习题 8.6** 编写一个叫做 myecho 的程序，打印出它的命令行参数和环境变量。例如：

```
linux> ./myecho arg1 arg2
```

```
Command-line arguments:
```

```
argv[ 0]: myecho
```

```
argv[ 1]: arg1
```

```
argv[ 2]: arg2
```

```
Environment variables:
```

```
envp[ 0]: PWD=/usr0/droh/ics/code/ecf
```

```
envp[ 1]: TERM=emacs
```

```
:
```

```
:
```

```
envp[25]: USER=droh
```

```
envp[26]: SHELL=/usr/local/bin/tcsh
```

```
envp[27]: HOME=/usr0/droh
```

8.4.6 利用 fork 和 execve 运行程序

像 Unix shell 和 Web 服务器这样的程序大量使用了 fork 和 execve 函数。shell 是一个交互型的应用级程序，它代表用户运行其他程序。最早的 shell 是 sh 程序，后面出现了一些变种，比如 csh、tcsh、ksh 和 bash。shell 执行一系列的读/求值(read/evaluate)步骤，然后终止。读步骤读取来自用户的一个命令行。求值步骤解析命令行，并代表用户运行程序。

图 8-23 展示了一个简单 shell 的 main 例程。shell 打印一个命令行提示符，等待用户在 stdin 上输入命令行，然后对这个命令行求值。

```

code/ecf/shellex.c

1  #include "csapp.h"
2  #define MAXARGS  128
3
4  /* Function prototypes */
5  void eval(char *cmdline);
6  int parseline(char *buf, char **argv);
7  int builtin_command(char **argv);
8
9  int main()
10 {
11     char cmdline[MAXLINE]; /* Command line */
12
13     while (1) {
14         /* Read */
15         printf("> ");
16         Fgets(cmdline, MAXLINE, stdin);
17         if (feof(stdin))
18             exit(0);
19
20         /* Evaluate */
21         eval(cmdline);
22     }
23 }
```

code/ecf/shellex.c

图 8-23 一个简单的 shell 程序的 main 例程

图 8-24 展示了对命令行求值的代码。它的首要任务是调用 parseline 函数(见图 8-25)，这个函数解析了以空格分隔的命令行参数，并构造最终会传递给 execve 的 argv 向量。第一个参数被假设为要么是一个内置的 shell 命令名，马上就会解释这个命令，要么是一个可执行目标文件，会在一个新的子进程的上下文中加载并运行这个文件。

如果最后一个参数是一个“&”字符，那么 parseline 返回 1，表示应该在后台执行该程序(shell 不会等待它完成)。否则，它返回 0，表示应该在前台执行这个程序(shell 会等待它完成)。

在解析了命令行之后，eval 函数调用 builtin_command 函数，该函数检查第一个命令行参数是否是一个内置的 shell 命令。如果是，它就立即解释这个命令，并返回值 1。否则返回 0。简单的 shell 只有一个内置命令——quit 命令，该命令会终止 shell。实际使用

的 shell 有大量的命令，比如 `pwd`、`jobs` 和 `fg`。

如果 `builtin_command` 返回 0，那么 shell 创建一个子进程，并在子进程中执行所请求的程序。如果用户要求在后台运行该程序，那么 shell 返回到循环的顶部，等待下一个命令行。否则，shell 使用 `waitpid` 函数等待作业终止。当作业终止时，shell 就开始下一轮迭代。

code/ecf/shellex.c

```

1  /* eval - Evaluate a command line */
2  void eval(char *cmdline)
3  {
4      char *argv[MAXARGS]; /* Argument list execve() */
5      char buf[MAXLINE];   /* Holds modified command line */
6      int bg;              /* Should the job run in bg or fg? */
7      pid_t pid;           /* Process id */
8
9      strcpy(buf, cmdline);
10     bg = parseline(buf, argv);
11     if (argv[0] == NULL)
12         return; /* Ignore empty lines */
13
14     if (!builtin_command(argv)) {
15         if ((pid = Fork()) == 0) { /* Child runs user job */
16             if (execve(argv[0], argv, environ) < 0) {
17                 printf("%s: Command not found.\n", argv[0]);
18                 exit(0);
19             }
20         }
21
22         /* Parent waits for foreground job to terminate */
23         if (!bg) {
24             int status;
25             if (waitpid(pid, &status, 0) < 0)
26                 unix_error("waitfg: waitpid error");
27         }
28         else
29             printf("%d %s", pid, cmdline);
30     }
31     return;
32 }
33
34 /* If first arg is a builtin command, run it and return true */
35 int builtin_command(char **argv)
36 {
37     if (!strcmp(argv[0], "quit")) /* quit command */
38         exit(0);
39     if (!strcmp(argv[0], "&")) /* Ignore singleton & */
40         return 1;
41     return 0; /* Not a builtin command */
42 }

```

code/ecf/shellex.c

图 8-24 eval 对 shell 命令行求值

code/ecf/shellex.c

```
1  /* parseline - Parse the command line and build the argv array */
2  int parseline(char *buf, char **argv)
3  {
4      char *delim;          /* Points to first space delimiter */
5      int argc;             /* Number of args */
6      int bg;               /* Background job? */
7
8      buf[strlen(buf)-1] = ' '; /* Replace trailing '\n' with space */
9      while (*buf && (*buf == ' ')) /* Ignore leading spaces */
10         buf++;
11
12     /* Build the argv list */
13     argc = 0;
14     while ((delim = strchr(buf, ' ')) {
15         argv[argc++] = buf;
16         *delim = '\0';
17         buf = delim + 1;
18         while (*buf && (*buf == ' ')) /* Ignore spaces */
19             buf++;
20     }
21     argv[argc] = NULL;
22
23     if (argc == 0) /* Ignore blank line */
24         return 1;
25
26     /* Should the job run in the background? */
27     if ((bg = (*argv[argc-1] == '&')) != 0)
28         argv[--argc] = NULL;
29
30     return bg;
31 }
```

code/ecf/shellex.c

图 8-25 parseline 解析 shell 的一个输入行

注意，这个简单的 shell 是有缺陷的，因为它并不回收它的后台子进程。修改这个缺陷就要求使用信号，我们将在下一节中讲述信号。

8.5 信号

到目前为止对异常控制流的学习中，我们已经看到了硬件和软件是如何合作以提供基本的低层异常机制的。我们也看到了操作系统如何利用异常来支持进程上下文切换的异常控制流形式。在本节中，我们将研究一种更高层的软件形式的异常，称为 Linux 信号，它允许进程和内核中断其他进程。

一个信号就是一条小消息，它通知进程系统中发生了一个某种类型的事件。比如，图 8-26 展示了 Linux 系统上支持的 30 种不同类型的信号。

每种信号类型都对应于某种系统事件。低层的硬件异常是由内核异常处理程序处理的，正常情况下，对用户进程而言是不可见的。信号提供了一种机制，通知用户进程发生了这些异常。比如，如果一个进程试图除以 0，那么内核就发送给它一个 SIGFPE 信号(号码 8)。如果一个进